PB UNDERGROUND

Technical Audit Report

Repository, frontend, backend, database, connectivity and production-readiness review

---

AUDIT SNAPSHOT

25 October 2026

Repository: github.com/sabyasachi69/PB-UNDERGROUND

Branch: main / Commit: bb4b861 / Working tree: clean

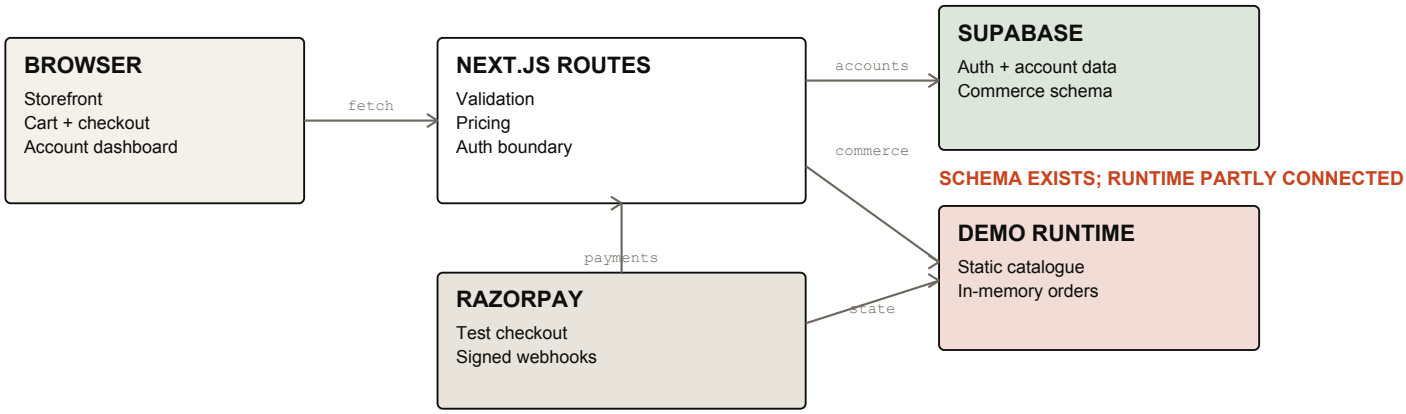
Prepared from the implementation repository and PB UNDERGROUND Website Build Specification v1.0. No source code was changed during this review.

AUDIT CONCLUSION

00. Executive assessment

PB UNDERGROUND is a strong Phase 0-5 demonstration, but it is not yet a fully connected production commerce system.

The frontend experience, account flows, validation, demo checkout and database design are substantial. The principal architectural limitation is that the application currently has two partially separate systems: a polished demo commerce runtime and a Supabase-backed customer-account runtime.



- The demo works well while one server process remains running.
- Authentication and customer account records can use Supabase.
- Catalogue, inventory, orders, tracking and payment state do not yet use the Supabase commerce tables.
- A restart or serverless instance change loses demo orders, inventory changes, sessions and webhook replay state.
- Phase 6 and later phases have not been built.
- Several earlier phases satisfy their functional demo goals but not every production acceptance criterion in the specification.

Connectivity matrix

Area	Current source	Durability	Assessment
Catalogue	Checked-in TypeScript fixtures	Versioned, not editable	Good demo; database is unused
Cart	Browser localStorage	One browser	Suitable for demo only
Checkout + orders	Process-memory store	Lost on restart	Major production blocker
Customer accounts	Demo store or Supabase	Durable when connected	Most complete integration
Payments	Razorpay test + memory state	Provider external; app state volatile	Signatures good; persistence incomplete

IMPLEMENTED CAPABILITY

01. What is done

A. Repository and engineering foundation

The project has a credible modern foundation: Next.js 16.4.2 App Router, React 19.3.1, strict TypeScript, ESLint Core Web Vitals rules, Vitest, Supabase CLI migrations, GitHub Actions, an environment-variable contract and explicit architectural decision records.

- CI runs npm ci, lint, type-check, unit tests and a production build on pushes to main and on pull requests.
- Secrets are excluded from source control and .env.example lists the expected variables without values.
- Technical assumptions and reversible business defaults are recorded in DECISIONS.md.
- The repository uses Server Components by default and client components only for interactive workflows.
- Money values are represented as integer paise.

Evidence: .github/workflows/ci.yml / README.md / DECISIONS.md

B. Brand and storefront frontend

- Eight-section editorial home page: hero, manifesto, journey, drops, delivery, partners, voices and closing CTA.
- Custom typography, paper/ink/signal palette, responsive layouts and clearly labelled sample content.
- Navigation, footer, About page, reduced-motion treatment and skip-to-content support.
- Product/category metadata, canonical URLs and Product JSON-LD.
- Custom not-found route and branded loading, empty and error states across primary commerce flows.

The intended narrative structure is present. The specification's advanced motion system - scroll-scrubbed text, pinned horizontal storytelling, interactive map, carousel and magnetic CTA - has mostly been simplified to CSS transitions and entrance animation. The design foundation is polished, but the exact award-level motion acceptance criterion is not evidenced.

C. Catalogue

- Twelve sample products across four categories.
- Product and category routes, search, URL-synchronized filters and sorting.
- Size, colour, supplier, price and stock filters.
- Variant stock, out-of-stock states, quick add, wishlist controls, gallery, zoom and size guide.
- Related products, sample reviews and inclusive-GST price presentation.

The current catalogue filters checked-in data directly in the client. Public product APIs exist, but the storefront does not use them as its primary data source. Evidence: src/components/catalogue-client.tsx

D. Cart and checkout

- Persistent browser cart with quantity changes, remove and save-for-later.
- Server-calculated quotes, coupon validation and stock checks.
- Delivery fee preview, free-delivery progress and minimum-order enforcement.
- Pincode/zone validation and cutoff-aware delivery slots.
- Three-step guest checkout with saved-address support for signed-in users.
- COD, UPI test integration and a clearly labelled local payment simulator.
- Client-generated idempotency key, order confirmation and live-style tracking page.

The cart itself is stored in localStorage. Prices and stock are nevertheless recalculated on the server from trusted sample data, so browser totals are not accepted as authoritative. Evidence: src/lib/cart.ts / src/lib/commerce.ts

E. Customer accounts

- Registration, login, sign-out and Supabase Auth integration when configured.
- Credential-free local authentication using scrypt and a signed HttpOnly SameSite=Lax cookie.
- Profile editing and up to five saved addresses.
- Order history, one-click reorder and variant wishlists.
- Photo-backed return requests within a 48-hour eligibility window.
- Confirmed account deletion.
- Private return-evidence bucket with JPEG, PNG and WebP restrictions and a 2 MB object limit.

The Supabase account repository loads addresses, wishlists, orders and returns concurrently, while the demo adapter provides equivalent workflows for local review. Evidence: src/lib/account-repository.ts / src/lib/auth.ts

F. Backend and APIs

Implemented route groups cover authentication, customer accounts, catalogue reads, cart quotes, coupons, zones, slots, order creation, tracking, payment verification, payment failure, Razorpay webhooks and the local payment simulator.

- Zod validation at the main request boundaries.
- Same-origin checks on state-changing routes.
- Basic per-process rate limiting.
- Timing-safe comparison for passwords and HMAC signatures.
- Raw-body Razorpay webhook verification and event-ID replay detection.
- Payment amount and currency checks, order idempotency and forward-only order transitions.
- No card-number storage; payment collection remains Razorpay-hosted.

G. Database foundation

Domain	Implemented tables and controls
Catalogue	suppliers, categories, products, product_variants, product_images, reviews, stock_notifications; unique slugs/SKUs, publication states, price and alt-text checks, public read RLS
Commerce	zones, delivery_windows, coupons, carts, cart_items, orders, order_items, order_events, settings; snapshots, idempotency, totals and status constraints
Payments	payments, inventory_reservations, payment_webhook_events; provider uniqueness, replay records and atomic expiry release function
Accounts	profiles, addresses, wishlists, returns; auth trigger, five-address trigger, ownership RLS and private evidence storage

The database design is one of the strongest parts of the project. It provides a credible target architecture even though the main commerce runtime does not yet use it. Evidence: supabase/migrations/20261023094206_catalogue_foundation.sql / supabase/migrations/20261023142704_checkout_foundation.sql / supabase/migrations/20261023144220_payments_foundation.sql / supabase/migrations/20261023190915_accounts_foundation.sql

H. Verification snapshot

Check	Result	Interpretation
Git working tree	Clean	No uncommitted changes
Local vs origin/main	bb4b861	Synchronized
Production build	Passed	49 generated application routes/pages
TypeScript	Passed	Strict compile succeeds
ESLint	Passed	No warnings
Unit tests	22 passed / 2 failed	Date-dependent payment tests
npm audit --omit=dev	0 vulnerabilities	Current production dependency result
Playwright / axe / Lighthouse	Not present	Acceptance evidence still required

DEVELOPMENT RECORD

02. Problems faced and how they were handled

Resolved during development

Next.js 16 API differences

The project uses a newer Next.js release with changed asynchronous APIs and conventions. Installed framework documentation was checked; async cookies and route parameters were implemented correctly, and the production build succeeds.

No production Supabase credentials

A dual runtime was built. Local review uses isolated in-memory accounts and orders; configured environments switch customer authentication and account records to Supabase. This kept development unblocked, although the fallback architecture is now the main production connectivity gap.

No Razorpay test credentials

A local success/failure payment simulator was added. Real Razorpay test order creation activates only when all required test credentials exist, and live keys are deliberately rejected.

Missing real products and photography

Twelve coherent sample products and four fictional suppliers were introduced. User-facing suppliers, reviews, visuals and statistics are marked SAMPLE rather than presented as real.

Credential fields entering profile snapshots

Account QA found that a structural object spread could copy internal demo authentication fields. Profile creation now explicitly projects safe fields, and a regression test verifies that salts and password hashes cannot enter account snapshots.

Ownership and address limits

Owner filtering, RLS policies, database triggers and tests were combined so direct API or database calls cannot bypass the intended five-address and customer-isolation rules.

Active problems discovered in this audit

1. Two payment tests are date-dependent and currently fail

The payment tests use fixed timestamps from 24 October 2026, while several helper calls fall back to the real current time. On 25 October, those synthetic reservations are seen as expired. As a result, npm run check is currently red even though lint and TypeScript pass.

Required response: inject a clock into every payment-store helper, use relative test times or fake timers, and reset global demo state before every test. Evidence: tests/payments.test.ts

2. Supabase commerce tables are not connected to order creation

The order route always invokes createDemoOrder. Orders, order items, reservations, payments and webhook events are therefore not inserted into Supabase. Database inventory is not decremented, and a Supabase-authenticated customer's in-memory order will not appear in the account dashboard that reads orders from Supabase.

Evidence: src/app/api/orders/route.ts

3. Variant identifiers are incompatible across runtimes

The static catalogue uses readable string IDs. Supabase uses UUIDs. Account snapshots return UUIDs, but reorder and wishlist-to-cart insert those UUIDs into a cart whose quote engine only recognizes the static string IDs. These flows work in demo mode and fail after full Supabase connection.

4. Public tracking uses only a predictable order number

Tracking requires no token or secondary verification. Order numbers have a date and four-digit suffix, while the response includes first name, area, purchased items, sizes, totals and payment state. A random tracking token or phone verification should protect this endpoint.

5. Durable payment design exists only in SQL

The schema includes durable reservations and webhook replay records, but runtime provider mappings and event IDs live in process memory. In serverless hosting, checkout may run on one instance and the webhook on another. A restart also discards payment context and replay protection.

PRIORITIZED ROADMAP

03. Scope and areas of improvement

Recommended sequence: stabilize the integration boundary before building the supplier portal on top of temporary order state.

Priority 0 - required before real transactions

Build a single durable commerce repository

Replace direct dependence on static products and the in-memory order store with explicit Supabase and demo adapters using one canonical model.

Make order creation transactional

Lock variants, verify supplier/product state, validate stock and coupons, recalculate totals, reserve/decrement stock, insert the order/items/payment/event and commit atomically.

Make Razorpay processing durable

Resolve provider order IDs from the payments table, deduplicate webhooks with a database constraint and update payment, order and inventory states in one transaction.

Fix canonical variant IDs

Use Supabase UUIDs throughout, or introduce a stable public variant key/SKU and translate it only on the server.

Protect tracking links

Require a high-entropy tracking token or secondary verification and return a deliberately minimized tracking DTO.

Repair the test suite

Inject time, reset shared stores, add route-level integration coverage and return CI to green before declaring another phase complete.

Priority 1 - database and account integrity

- Make default-address replacement atomic so an insert failure cannot leave a customer without a default.
- Verify every returned item belongs to the selected order and that quantities do not exceed purchased quantities.
- Support item-level and partial returns rather than one request per whole order.
- Clean up uploaded evidence when a return insert fails, and delete objects during account deletion where policy requires it.
- Validate image magic bytes and use direct signed uploads instead of large base64 JSON bodies.
- Document retention and anonymization rules for order snapshots after account deletion.
- Load zones, fees, delivery windows and coupons from the settings/database layer instead of hard-coded constants.

Missing specification entities

Still absent from the specification model: refunds and item-level refund records; settlements and payout statements; supplier, staff and owner roles; supplier profile ownership; notifications; banners; an append-only audit_log; and newsletter_subscribers.

Priority 2 - supplier Phase 6

Phase 6 has not started. Its data and authorization model should be built only after durable commerce connectivity is in place.

- /partner sales page and live earnings calculator
- Supplier registration, business profile and admin approval
- Supplier dashboard, onboarding checklist and account settings
- Product, variant, image and stock manager
- Supplier-specific order queue with a 30-minute accept/decline window
- Packing, issue reporting, pickup and handoff states
- Weekly earnings and settlement pages with downloadable CSV
- Supplier-specific RLS and automated cross-supplier isolation tests

The supplier model must gain auth profile ownership, GSTIN, business and pickup addresses, pickup hours, masked bank reference and approval information.

Priority 3 - frontend and content quality

- Complete the scroll-scrubbed manifesto, pinned journey, interactive zone map, swipe/focus-aware testimonial carousel and magnetic CTA system.
- Replace abstract sample visuals with real optimized product and supplier photography.
- Correct stale order-tracking copy that says account features arrive in Phase 5.
- Add Playwright and axe checks, modal focus trapping, Escape behavior, focus restoration and field-error associations.
- Verify touch targets, keyboard-only checkout, colour contrast and all specified viewport sizes.
- Record repeatable mobile Lighthouse results for home, shop and product routes.

Priority 4 - security and operations

Current headers include clickjacking, MIME sniffing, referrer and permissions protections. Production hardening still needs:

- Content-Security-Policy and production HSTS
- Razorpay script nonces or a tightly scoped script policy
- Distributed rate limiting and trusted proxy/IP configuration
- Signup abuse controls
- Structured logs and error monitoring
- Payment reconciliation and scheduled reservation expiry
- Backup/export procedures and secret-rotation documentation

Evidence: next.config.ts / src/lib/api-security.ts

Priority 5 - remaining specification routes

Not yet present: /delivery, /contact, /legal/*, /partner, /partner/demo, /supplier/*, /admin/*, branded /500, sitemap.xml, robots.txt, newsletter, contact form, email order confirmations, monitoring and analytics.

IMPLEMENTATION DECISION

04. Recommended next move

Run an integration and stabilization pass before starting the supplier interface.

01	Repair the payment tests Remove calendar dependence, inject time and isolate shared demo state.
02	Unify product and variant identifiers Ensure catalogue, cart, wishlist, reorder and database records use one canonical key model.
03	Connect commerce to Supabase Implement durable catalogue reads, transactional order creation, inventory reservations and persisted order events.
04	Persist payment processing Move provider mapping, webhook replay protection and payment transitions into the database.
05	Secure tracking Add a high-entropy token or secondary verification and return a minimized projection.
06	Add one end-to-end COD test Prove browse, cart, checkout, database order creation and account history as one connected flow.
07	Start Phase 6 Build supplier onboarding, products, orders and settlements on the durable model.

Why this sequence

Building the supplier portal directly against the temporary in-memory order system would create a second set of product, order and payout assumptions that later need to be rewritten. Stabilizing the shared commerce layer first allows both customer and supplier experiences to operate on the same durable source of truth.

Final assessment

PB UNDERGROUND has progressed well beyond a visual prototype. It has a distinctive storefront, thoughtful interaction states, meaningful server-side validation, a credible database design and a functioning customer-account demonstration. The next engineering milestone is not more surface area; it is joining the existing parts into one durable system. Once catalogue, orders, inventory and payments are truly database-backed, the project will have the right foundation for Phase 6 and the eventual admin and launch-hardening phases.

APPENDIX

A. Evidence index

This report was prepared from the original website specification, the repository at commit bb4b861, direct source inspection and local verification performed on 25 October 2026.

Evidence area	Primary source
Project status and phase claims	docs/IMPLEMENTATION_PLAN.md
Setup and runtime contract	README.md / .env.example
Architecture decisions	DECISIONS.md
Authentication	src/lib/auth.ts
Account persistence	src/lib/account-repository.ts
Order runtime	src/app/api/orders/route.ts / src/lib/demo-order-store.ts
Pricing and delivery	src/lib/commerce.ts
Security boundaries	src/lib/api-security.ts / next.config.ts
Database migrations	supabase/migrations/20261023190915_accounts_foundation.sql
Current test defect	tests/payments.test.ts

Verification commands

```
npm run check
npm run build
npm audit --omit=dev
```

Repository: github.com/sabyasachi69/PB-UNDERGROUND

Scope note: this was a read-only implementation audit. It did not alter application source, migrations or repository history.